

Guia de implementação do módulo comercial do Grupo Grapiúna

Escopo: pacotes, benefícios, conteúdos/serviços, limites de publicidade, clientes, contratos, parcelas, recebimentos, relatórios, usuários e pré-visualização.

Base analisada: branch `main` do repositório `FelipeSystem/grupograpiuna`, no commit `179f736`.

1. Diagnóstico antes de começar

A primeira decisão importante é não aplicar diretamente o README enviado sobre o repositório atual. O README anexado descreve um painel administrativo construído com Next.js, PostgreSQL, migrações, seed e autenticação por senha, enquanto o repositório público analisado é uma SPA em React com Vite, servidor Express auxiliar, Firebase Authentication e Cloud Firestore. Isso aparece no `package.json`, que não possui Next.js, PostgreSQL, Drizzle, bcrypt ou scripts de migração, e também no código de inicialização do Firebase. 1 2

O painel existente é predominantemente editorial. `AdminDashboard.tsx` possui as abas Audiência, Notícias, TV Grade, Canais TV, Publicidade, Hub73, Podcasts, Vídeos, Stories e Loja, mas não possui abas de clientes, pacotes, contratos, financeiro, relatórios comerciais ou usuários. O acesso também está limitado diretamente a `admin` e `editor`. 3

A publicidade já possui os cinco formatos pedidos: `cover`, `leaderboard`, `intermediario`, `sidebar` e `mobile`. Entretanto, o cadastro atual grava apenas tamanho, imagem, link, página e status na coleção `ads`; não há relação com cliente, contrato ou pacote. 4 O componente público `AdBanner` consulta essa coleção, filtra por formato e status e exibe um banner por slot, usando o índice para alternar entre anúncios. Portanto, os limites de um pacote ainda não são aplicados pela aplicação. 5

As regras do Firestore confirmam a lacuna: existem permissões para `news`, `tv_schedule`, `tv_channels`, `ads`, `users`, `podcasts`, `youtube_videos` e outras coleções editoriais, mas não há entidades comerciais para pacotes, clientes, contratos, parcelas, pagamentos ou alocações de anúncios. 6 A página pública `/anuncie` é apenas uma apresentação visual; o formulário de solicitação não possui `onSubmit`, estado React nem gravação em banco. 7

A base atual está saudável para receber uma extensão: depois de instalar as dependências, `npm run lint` e `npm run build` foram executados com sucesso. O build emite apenas um aviso de bundle grande, sem impedir a compilação. A conclusão prática é a seguinte: **a forma mais segura e incremental é implementar o módulo comercial sobre React/Vite + Firebase, mantendo o portal editorial existente.** PostgreSQL só deve ser escolhido se for

um requisito obrigatório, pois nesse caso será uma migração arquitetural, não uma simples alteração de tela.

Item	Estado encontrado	Consequência para a implementação
Frontend	React 19 + Vite + TypeScript	Criar componentes React dentro de <code>src/components</code> e rotas com <code>react-router-dom</code> .
Backend atual	Endpoints serverless em <code>api/</code> e servidor Express em <code>server.ts</code>	Criar endpoints server-side para operações sensíveis, sem colocar credenciais no bundle do navegador.
Banco atual	Cloud Firestore	Criar coleções e regras novas; não executar <code>db:migrate</code> ou <code>db:seed</code> do README anexado.
Autenticação	Firebase Auth, atualmente com Google	Manter login Google para a equipe ou adicionar e-mail/senha pelo Firebase Auth; não guardar senha em documento Firestore.
Papéis atuais	<code>admin</code> , <code>editor</code> e fallback <code>user</code>	Migrar para <code>admin</code> , <code>gerente</code> , <code>comercial</code> , <code>financeiro</code> , <code>operador_anuncios</code> e <code>visualizacao</code> , mantendo compatibilidade temporária com <code>editor</code> .
Publicidade	Coleção pública <code>ads</code> e componente <code>AdBanner</code>	Preservar os anúncios editoriais atuais e criar uma camada privada de alocações comerciais.
Página <code>/anuncie</code>	Formulário visual sem persistência	Transformá-la em catálogo público de pacotes e/ou formulário operacional de solicitação.

2. Decisão de arquitetura recomendada

O módulo deve ser dividido em quatro responsabilidades. A primeira é o **catálogo comercial**, que contém pacotes, preços por duração, benefícios e serviços. A segunda é o

relacionamento comercial, que contém clientes, contratos e o retrato do pacote no momento da venda. A terceira é a **entrega de publicidade**, que controla anúncios, formatos e cotas. A quarta é o **financeiro**, que contém parcelas, pagamentos, situação de vencimento e relatórios.

Essa separação é necessária porque um pacote pode ser alterado depois de vendido. O contrato não deve consultar apenas o pacote atual, pois isso faria uma alteração futura de preço ou de limite mudar retroativamente o que já foi vendido. Ao fechar o contrato, grave no próprio contrato um `packageSnapshot` com nome, preço escolhido, duração, benefícios, conteúdos e limites de anúncios vigentes naquele momento.

O Firestore é orientado a documentos e coleções, permite mapas e subcoleções e não exige a criação formal de tabelas antes da gravação. Subcoleções são adequadas para dados que crescem associados a um documento, como benefícios e serviços de um pacote. ⁸ Para consistência entre contrato, parcelas e contadores de utilização, utilize transações ou gravações em lote; essas operações são atômicas e não aplicam somente parte das alterações quando falham. ¹⁰

Camada	Responsabilidade	Onde implementar
Catálogo	Pacotes, preços, benefícios e serviços	Firestore + <code>AdminPackages.tsx</code>
Vendas	Clientes, contratos, vendedor, duração e parcelas iniciais	API server-side + <code>AdminClients.tsx</code> e <code>AdminContracts.tsx</code>
Operação	Upload, aprovação e publicação dos anúncios contratados	<code>AdminClientAds.tsx</code> + coleção privada e projeção pública
Financeiro	Pendências, atrasos, baixa e histórico de pagamentos	API server-side + <code>AdminFinance.tsx</code>
Gestão	Usuários, papéis, permissões e auditoria	Firebase Auth + <code>users</code> + <code>activity_logs</code>
Indicadores	Cards e relatórios comerciais	<code>AdminCommercialDashboard.tsx</code> e <code>AdminReports.tsx</code>

3. Modelo de dados no Firestore

3.1 Coleção `packages`

Crie um documento por pacote em `packages/{packageId}` . Os quatro períodos devem ficar no mesmo documento porque são atributos do pacote e porque a tela de pré-visualização precisará alternar entre eles sem fazer quatro consultas.

Plain Text

```
export type PackageDuration = 1 | 3 | 6 | 12;

export type AdFormat =
  | 'cover'
  | 'leaderboard'
  | 'intermediario'
  | 'sidebar'
  | 'mobile';

export interface PackageDocument {
  name: string;
  slug: string;
  description: string;
  color: string; // hex, por exemplo #B87333
  isActive: boolean;
  isFeatured: boolean;
  sortOrder: number;
  prices: Record<PackageDuration, number>; // armazenar em centavos
  adLimits: Record<AdFormat, number>;
  createdAt: Timestamp;
  updatedAt: Timestamp;
  createdBy: string;
  updatedBy: string;
}
```

Armazene valores monetários em centavos inteiros, e não em `number` decimal com ponto flutuante. Assim, `R$ 1.250,00` fica como `125000` . O formato de apresentação deve ser aplicado somente na interface, com `Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' })` .

Os limites sugeridos no pedido podem ser representados da seguinte maneira. Os valores de Prata e Master que não foram informados devem ser decididos pelo Grupo Grapiúna antes do seed; não é recomendável inventá-los no código.

Pacote	Capa TV	Leaderboard	Intermediário	Sidebar	Mobile
Bronze	0	0	0	1	1
Prata	definir	definir	definir	definir	definir
Ouro	1	2	0 ou definir	2	4

Master	definir	definir	definir	definir	definir
--------	---------	---------	---------	---------	---------

O valor `0` significa que o formato não está incluído. Se o negócio quiser “ilimitado”, não use `-1` sem documentar; prefira um campo explícito como `unlimitedFormats: AdFormat[]` ou mantenha o limite numérico com uma regra clara. Para a primeira versão, limites finitos são mais simples de auditar.

3.2 Benefícios e conteúdos/serviços

Use duas subcoleções porque os conceitos têm comportamento diferente. Benefício é normalmente um texto comercial livre, enquanto conteúdo é um serviço que pode ser selecionado de um catálogo, receber quantidade e ser controlado operacionalmente.

Plain Text

```
packages/{packageId}/benefits/{benefitId}
  label: string
  order: number
  isActive: boolean
  createdAt: Timestamp

packages/{packageId}/contents/{contentId}
  serviceId: string | null
  title: string
  type: 'materia' | 'redes_sociais' | 'cobertura_evento' |
        'programa' | 'video' | 'banner_portal' | 'outro'
  description: string
  quantity: number | null
  order: number
  isActive: boolean
  createdAt: Timestamp
```

Crie também `service_catalog/{serviceId}` para que serviços reutilizáveis não precisem ser digitados novamente em cada pacote. Os registros podem representar publicação de matéria, divulgação em redes sociais, cobertura de evento, participação em programa, inserção em vídeo, banner no portal e novos serviços que forem criados no futuro.

Campo do catálogo	Exemplo
<code>name</code>	Cobertura de evento
<code>type</code>	<code>cobertura_evento</code>
<code>description</code>	Equipe registra e publica conteúdo do evento

isActive	true
defaultQuantity	1 ou null

3.3 Clientes

Crie `clients/{clientId}` com os dados cadastrais e comerciais, mas não grave dados de cartão ou senha. Para facilitar busca e relatórios, normalize e-mail e documento em campos auxiliares, sem remover a versão formatada que aparecerá na tela.

Plain Text

```
clients/{clientId}
  legalName: string
  tradeName: string
  documentType: 'cpf' | 'cnpj'
  documentNumber: string
  email: string
  phone: string
  whatsapp: string
  address: {
    street: string
    number: string
    complement: string
    neighborhood: string
    city: string
    state: string
    zipCode: string
  }
  contactName: string
  notes: string
  status: 'active' | 'inactive'
  createdAt: Timestamp
  updatedAt: Timestamp
  createdBy: string
```

O cadastro deve impedir duplicidade de CPF/CNPJ normalizado. Para isso, crie documentos de índice como `client_identifiers/{normalizedDocument}` apontando para o cliente, ou faça essa verificação numa API transacional. Uma consulta do frontend sozinha não é suficiente quando dois usuários salvam ao mesmo tempo.

3.4 Contratos e snapshot do pacote

Crie `contracts/{contractId}` . O contrato deve apontar para o pacote original e, simultaneamente, guardar o snapshot. Assim, relatórios históricos continuam corretos quando o cadastro do pacote mudar.

Plain Text

```
contracts/{contractId}
  clientId: string
  packageId: string
  packageSnapshot: {
    name: string
    color: string
    durationMonths: 1 | 3 | 6 | 12
    priceCents: number
    benefits: string[]
    contents: object[]
    adLimits: Record<AdFormat, number>
  }
  durationMonths: 1 | 3 | 6 | 12
  totalCents: number
  installmentCount: number
  firstDueDate: string          // YYYY-MM-DD, no fuso de negócio
  startDate: string
  endDate: string
  sellerId: string
  paymentMethod: 'pix' | 'boleto' | 'cartao' | 'dinheiro' | 'transferencia' | '
  status: 'draft' | 'pending' | 'active' | 'expired' | 'cancelled'
  createdAt: Timestamp
  updatedAt: Timestamp
  createdBy: string
```

O preço escolhido deve ser o preço do período contratado. Se o comercial quiser parcelar um contrato de 12 meses em 12 parcelas, `totalCents` é o preço de 12 meses e `installmentCount` é 12. Se houver desconto manual, grave `subtotalCents` , `discountCents` , `totalCents` e o motivo do desconto; não sobrescreva silenciosamente o preço do pacote.

3.5 Parcelas e pagamentos

Use uma coleção de topo `installments` para facilitar relatórios e filtros por vencimento. Cada parcela deve apontar para o contrato e para o cliente.

Plain Text

```
installments/{installmentId}
  contractId: string
  clientId: string
```

```
number: number
dueDate: string          // YYYY-MM-DD
amountCents: number
status: 'pending' | 'paid' | 'overdue' | 'cancelled'
paidCents: number
paidAt: Timestamp | null
paymentMethod: string | null
updatedAt: Timestamp

installments/{installmentId}/payments/{paymentId}
amountCents: number
paidAt: Timestamp
method: string
reference: string
notes: string
registeredBy: string
```

Mesmo que a primeira versão permita somente pagamento integral, a subcoleção de pagamentos evita remodelagem quando surgir pagamento parcial, estorno ou mais de uma forma de pagamento. O status pode ser recalculado na consulta, mas a baixa financeira deve ser escrita por uma API autenticada e auditada.

3.6 Limites e alocações de anúncios

Não exponha dados comerciais de clientes na coleção pública `ads`, pois as regras atuais permitem leitura pública dessa coleção. Mantenha uma coleção privada `client_ads` para a operação comercial e continue usando `ads` como projeção mínima para o componente público.

Plain Text

```
contracts/{contractId}/adUsage/{format}
format: AdFormat
limit: number
used: number
updatedAt: Timestamp

client_ads/{clientAdId}
clientId: string
contractId: string
format: AdFormat
imageUrl: string
link: string
page: string
status: 'draft' | 'waiting_review' | 'approved' | 'published' | 'paused' | 'e
publicAdId: string | null
createdAt: Timestamp
```



```
updatedAt: Timestamp

ads/{adId}
  size: AdFormat
  imageUrl: string
  link: string
  page: string
  active: boolean
  source: 'legacy' | 'contract'
  allocationId: string | null
  createdAt: Timestamp
  updatedAt: Timestamp
```

O campo `used` em `adUsage` representa alocações ativas, não impressões ou visualizações. No momento de criar uma alocação, uma transação deve ler o contrato e o contador, rejeitar a operação quando `used >= limit`, criar o `client_ads` e incrementar o contador. Na desativação, a mesma transação deve reduzir o contador. A regra precisa existir no servidor para evitar que duas abas ultrapassem a cota simultaneamente.

O componente `AdBanner` atual pode continuar exibindo os anúncios legados. Depois que a operação de publicação estiver pronta, a API cria ou atualiza a projeção em `ads`; assim o portal não precisa conhecer `clientId`, valor do contrato ou outras informações privadas. O prop `index` existente permite renderizar mais de uma unidade do mesmo formato quando a página realmente precisar mostrar vários anúncios ao mesmo tempo. 5

4. Preparação do projeto e primeira sequência de commits

Comece em uma branch própria, preservando a base atual e registrando o estado de compilação.

Bash

```
git checkout main
git pull origin main
git checkout -b feature/modulo-comercial
npm ci
npm run lint
npm run build
git add .
git commit -m "chore: baseline do portal antes do modulo comercial"
```

Instale as dependências que serão usadas no frontend, nas APIs e nos testes. O SDK administrativo deve ser importado somente em código server-side, nunca em

componentes React enviados ao navegador.

Bash

```
npm install zod firebase-admin
npm install -D firebase-tools @firebase/rules-unit-testing
```

Crie a seguinte estrutura inicial:

Plain Text

```
src/
  components/
    AdminPackages.tsx
    AdminClients.tsx
    AdminContracts.tsx
    AdminClientAds.tsx
    AdminFinance.tsx
    AdminReports.tsx
    AdminUsers.tsx
    AdminCommercialDashboard.tsx
    PackageForm.tsx
    PackagePreview.tsx
    PackageBenefitsEditor.tsx
    PackageContentsEditor.tsx
    PackageAdLimitsEditor.tsx
  lib/
    commercial-types.ts
    commercial-validation.ts
    commercial-formatters.ts
    installment-dates.ts
    permissions.ts
  api/
    _lib/
      firebase-admin.ts
      require-auth.ts
      http.ts
    commercial/
      close-contract.ts
      assign-ad.ts
      register-payment.ts
      users.ts
  scripts/
    seed-commercial.mjs
    migrate-commercial.mjs
  firestore.indexes.json
```

O primeiro commit de código deve conter apenas tipos, validações e formatadores. O segundo deve conter regras e índices. Em seguida, implemente catálogo, contratos, financeiro, publicidade e relatórios em commits separados. Essa ordem permite revisar o modelo de dados sem misturar uma tela visual com uma operação financeira.

5. Implementação da tela de pacotes em duas camadas

Passo 1 — Criar a aba no painel

Altere `AdminDashboard.tsx` para adicionar uma aba `pacotes` e importar `AdminPackages`. Não coloque o componente dentro do bloco de publicidade, porque o cadastro de pacote é um catálogo comercial e não um cadastro de banner.

O tipo atual de `activeTab` é uma união literal que precisa ser ampliada. Também atualize o mapa de ícones e a renderização condicional. Na etapa seguinte, o mesmo painel receberá as abas Clientes, Contratos, Financeiro, Relatórios e Usuários.

TSX

```
const [activeTab, setActiveTab] = useState<  
  | 'analytics'  
  | 'packages'  
  | 'clients'  
  | 'contracts'  
  | 'clientAds'  
  | 'finance'  
  | 'reports'  
  | 'users'  
  | /* abas editoriais existentes */ string  
>('analytics' );
```

Não deixe a autorização somente no menu. O componente e as regras do Firestore também devem negar a operação. O Firebase documenta o uso de regras personalizadas para controle por papel; a regra deve consultar o usuário autenticado e validar a operação, em vez de depender de um botão oculto no frontend. ⁹

Passo 2 — Construir a primeira camada: Cadastro do Pacote

Crie um formulário controlado com os campos `name`, `description`, `color`, `prices`, `isActive` e `isFeatured`. O status deve aparecer como um controle claro de ativo/inativo. O destaque “Mais popular” deve ser booleano e não deve ser inferido pelo preço.

Na interface, apresente quatro cards de preço: 1 mês, 3 meses, 6 meses e 12 meses. Cada card recebe um valor em reais, mas converte para centavos antes de salvar. Não trate a

duração como uma linha de benefícios; ela é uma dimensão própria do preço.

Use Zod para validar antes do envio e repita as restrições nas regras ou na API. O frontend deve impedir preço negativo, duração inválida, limite decimal e cor fora do formato hexadecimal. A API deve repetir a validação porque qualquer cliente pode chamar uma rota HTTP diretamente.

Plain Text

```
const packageSchema = z.object({
  name: z.string().trim().min(2).max(80),
  description: z.string().trim().max(500),
  color: z.string().regex(/^#[0-9A-Fa-f]{6}$/),
  isActive: z.boolean(),
  isFeatured: z.boolean(),
  prices: z.object({
    1: z.number().int().nonnegative(),
    3: z.number().int().nonnegative(),
    6: z.number().int().nonnegative(),
    12: z.number().int().nonnegative(),
  }),
  adLimits: z.object({
    cover: z.number().int().nonnegative(),
    leaderboard: z.number().int().nonnegative(),
    intermediario: z.number().int().nonnegative(),
    sidebar: z.number().int().nonnegative(),
    mobile: z.number().int().nonnegative(),
  }),
});
```

Passo 3 — Construir a segunda camada: Benefícios, conteúdos e anúncios

Na mesma tela, use um segundo painel ou uma segunda etapa visual chamada “Conteúdos e benefícios do pacote”. Mantenha três blocos distintos: benefícios livres, serviços do catálogo e limites de formatos publicitários.

O editor de benefícios deve permitir adicionar, editar, reordenar e remover itens. O editor de conteúdos deve oferecer um select com os serviços ativos do `service_catalog`, além de permitir título customizado, descrição e quantidade. O editor de limites deve mostrar os nomes e dimensões já usados em `AdminAds.tsx`, para que o usuário não precise decorar `cover` ou `intermediario`.

Plain Text

Capa TV	1600 × 320 px	[0]
Leaderboard	970 × 90 px	[0]
Intermediário	728 × 90 px	[0]
Sidebar	300 × 250 px	[1]
Mobile	320 × 50 px	[1]

Na gravação, salve o documento de `packages` e as subcoleções com `writeBatch`. Em uma edição, leia os itens existentes, atualize os que permaneceram, crie os novos e apague os removidos. Não deixe benefícios excluídos apenas com `isActive: false` sem uma decisão clara, pois isso pode duplicar itens em uma pré-visualização futura.

Passo 4 — Construir a terceira camada: Pré-visualização

A pré-visualização deve consumir o mesmo estado validado do formulário, não uma segunda estrutura de dados. Ela deve apresentar o nome, a cor, a descrição, um seletor de período, o preço daquele período, benefícios, conteúdos e uma tabela de limites de anúncios.

O preview deve funcionar em desktop e mobile e ter dois modos: interno, que pode mostrar também status e campos de edição, e público, que mostra somente os dados comerciais que o Grupo Grapiúna decidiu publicar. Se o catálogo de pacotes for público, as regras devem liberar leitura apenas para documentos ativos; se for exclusivo da equipe, não deixe `allow read: if true` na coleção.

Passo 5 — Lista, edição e exclusão lógica

A lista de pacotes deve ordenar por `sortOrder` e nome, mostrar badges de status e destaque e permitir duplicar um pacote. A exclusão recomendada é lógica: marque `isActive: false` e preserve o pacote quando ele possuir contratos. Um pacote que já foi vendido não deve ser apagado fisicamente, pois o histórico depende do `packageSnapshot`.

Crie uma confirmação específica para desativação informando quantos contratos ativos usam o pacote. A operação pode continuar sendo permitida, mas o administrador precisa saber que novos contratos não poderão selecioná-lo.

6. Fechamento do pacote associado ao cliente

Passo 6 — Cadastro e seleção de cliente

Crie `AdminClients.tsx` com lista, busca por razão social/nome fantasia, CPF/CNPJ, e-mail e telefone. O botão “Novo contrato” deve abrir a seleção de um cliente existente ou permitir concluir o cadastro antes de avançar.

Ao selecionar o cliente, mostre contratos ativos, saldo pendente e anúncios alocados, mas não misture esses dados no documento de cliente. São informações derivadas de contratos e parcelas e devem ser consultadas separadamente ou retornadas por uma API de resumo.

Passo 7 — Modal ou rota de fechamento

Crie `AdminContracts.tsx` com um wizard de cinco etapas: cliente, pacote e duração, condições de pagamento, limites/entregas e revisão. O usuário deve escolher um pacote ativo e uma duração entre 1, 3, 6 e 12 meses. O preço deve vir do documento do pacote, e não de um campo digitado manualmente no navegador.

A etapa de revisão precisa mostrar claramente:

Informação	Exemplo de apresentação
Cliente	Empresa contratante e contato
Pacote	Ouro
Duração	12 meses
Total	R\$ valor do cadastro do pacote
Parcelas	12 × R\$ valor calculado
Limites	1 Capa TV, 2 Leaderboards, 2 Sidebars, 4 Mobiles
Benefícios	Lista do snapshot
Conteúdos	Serviços incluídos

Passo 8 — Endpoint transacional de fechamento

Não faça o fechamento escrevendo diretamente vários documentos a partir do componente React. Crie `api/commercial/close-contract.ts`, valide o token do Firebase, valide o payload com Zod, leia cliente e pacote no servidor, gere o snapshot e crie o contrato, as parcelas e os contadores de anúncios numa transação ou gravação em lote.

O fluxo recomendado é:

1. Verificar o token e o papel do usuário.
2. Verificar se o cliente existe e está ativo.
3. Verificar se o pacote existe, está ativo e possui preço para a duração escolhida.
4. Ler benefícios, conteúdos e catálogo necessários.

5. Calcular `totalCents` , número de parcelas e valor de cada parcela.
6. Gerar `startDate` , `endDate` e cada `dueDate` usando uma função de calendário testada.
7. Criar o contrato com `packageSnapshot` .
8. Criar de 1 a 12 parcelas, com status inicial `pending` .
9. Criar `adUsage/{format}` com `limit` do snapshot e `used: 0` .
10. Criar um registro de auditoria com usuário, data, ação e identificadores afetados.

O endpoint deve ser idempotente. Receba um `requestId` ou `idempotencyKey` e rejeite uma segunda solicitação com a mesma chave. Isso impede contratos duplicados quando o usuário clica duas vezes ou recarrega a página durante uma resposta lenta.

Uma função de geração de datas deve tratar o último dia do mês. Por exemplo, um vencimento mensal no dia 31 precisa ser ajustado para o último dia de fevereiro, abril, junho, setembro ou novembro. Não use simplesmente `new Date(year, month + 1, day)` sem testar esse caso.

7. Financeiro e baixa de pagamento

Crie `AdminFinance.tsx` com quatro visões: pendentes, pagos, atrasados e cancelados. A lista deve permitir filtrar por período, cliente, vendedor, pacote e status. O status atrasado é uma parcela não paga cujo vencimento passou no fuso de negócio do Grupo Gapiúna.

A baixa deve abrir um formulário com valor recebido, data, forma, referência e observação. Ao confirmar, `api/commercial/register-payment.ts` deve validar a parcela, criar o pagamento, atualizar `paidCents` , atualizar o status e registrar auditoria. Se houver pagamento parcial, mantenha a parcela `pending` até o valor total; se o vencimento passar antes da quitação, o status exibido deve ser `overdue` .

A operação de baixa deve ser transacional para impedir duas baixas concorrentes sobre a mesma parcela. O endpoint deve rejeitar valor zero, valor negativo, excesso não autorizado e parcela cancelada. Se o sistema precisar permitir estorno, crie uma operação inversa auditada em vez de editar silenciosamente o histórico original.

Para o dashboard, não dependa de baixar todas as parcelas para o navegador em uma única consulta. Mantenha resumos mensais como `financial_summaries/{YYYY-MM}` com totais contratados, recebidos, pendentes e atrasados, atualizados na mesma transação da baixa. Isso reduz custo e melhora o tempo de carregamento.

8. Aplicação prática da cota de publicidade

Passo 9 — Exibir limites no cliente

Na tela do cliente e do contrato, carregue o contrato ativo e apresente a tabela `limite / utilizado / disponível` por formato. O sistema deve usar o snapshot do contrato e os contadores `adUsage`, não o pacote corrente. Se o cliente tiver dois contratos ativos, defina uma regra explícita: usar somente o contrato vigente mais recente ou somar cotas; para a primeira versão, recomendo não permitir dois contratos ativos do mesmo cliente sem uma tela de consolidação.

Passo 10 — Criar e aprovar o anúncio do cliente

Crie `AdminClientAds.tsx` para que o operador selecione cliente, contrato, formato, página, imagem e link. A seleção de formato deve mostrar somente os formatos cuja disponibilidade é maior que zero. Mesmo assim, a API deve repetir essa verificação no servidor.

O ciclo recomendado é `draft → waiting_review → approved → published`, com estados `paused` e `expired` para operação. Na publicação, gere o documento mínimo em `ads` e guarde seu ID em `client_ads.publicAdId`. Na pausa, marque a projeção pública como inativa e reduza a utilização se a regra de negócio considerar que uma pausa libera a vaga.

O operador não deve alterar diretamente `adUsage.used` no frontend. A função `assign-ad.ts` deve ler o contador, comparar com o limite, criar a alocação e incrementar o uso no mesmo transaction. A documentação do Firebase recomenda transações para atualizações baseadas no valor corrente de outros documentos. 10

Passo 11 — Preservar compatibilidade com os anúncios existentes

Não substitua imediatamente a coleção `ads`, porque ela já é usada pela publicidade editorial do portal e pelo componente `AdBanner`. Adicione primeiro uma origem `legacy` ou `contract` e uma camada de projeção. Depois, atualize `AdminAds.tsx` para exibir uma seção “Anúncios do portal” e outra “Anúncios de contratos”.

Se o negócio quiser que um cliente tenha dois Leaderboards publicados ao mesmo tempo, renderize dois `AdBanner size="leaderboard" index={0|1}` nos slots públicos apropriados. O cadastro de dois anúncios no contrato não significa, por si só, que dois banners aparecerão no mesmo lugar: o portal atual exibe um por componente e alterna os registros. 5

9. Usuários, senha e papéis

Amplie `useAuth.ts` para devolver um papel tipado e funções de autorização, por exemplo `can('contracts.create')` e `can('finance.write')`. O painel atual verifica diretamente `role !== 'admin' && role !== 'editor'`; essa condição precisa ser substituída por uma matriz de permissões. 3

Papel	Pacotes	Clientes/Contratos	Financeiro	Anúncios	Usuários	Relatórios
Administrador	ler/escrever	ler/escrever/cancelar	ler/escrever/estornar	ler/escrever/publicar	ler/escrever	ler/escrever
Gerente	ler/escrever	ler/escrever/cancelar	ler	ler/aprovar	ler	ler
Comercial	ler	ler/escrever	ler	ler	não	ler comercial
Financeiro	ler	ler	ler/escrever/baixar	não	não	ler financeiro
Operador de anúncios	ler limites	ler cliente/contrato	não	ler/escrever/publicar	não	operacional
Visualização	ler	ler	ler	ler	não	ler

Mantenha `admin` e `editor` durante a migração. Uma opção é mapear `editor` temporariamente para `gerente`, registrar a conversão e remover o fallback depois de todos os usuários serem revisados. A regra deve consultar o documento `/users/{uid}` e validar operações por papel; esconder itens do menu não protege o banco. ⁹

Para senha, habilite o provedor Email/Password no Firebase Authentication e use `createUserWithEmailAndPassword` ou o fluxo de convite/reset. O Firebase armazena e valida as credenciais no serviço de autenticação; a aplicação não deve criar um campo `password` ou `passwordHash` em `users`. ¹¹ Para que um administrador crie usuários sem deslogar a própria conta, use uma API server-side com o Firebase Admin SDK e service account; a documentação oficial reserva a gestão elevada de usuários para um ambiente seguro de servidor. ¹²

Nunca coloque `firebase-admin`, service account, chave privada ou senha em arquivo iniciado por `VITE_`. Variáveis com esse prefixo fazem parte do bundle do navegador. Use apenas variáveis server-side em `api/_lib/firebase-admin.ts`.

10. Regras de segurança e índices

Atualize `firestore.rules` com funções de autenticação e papel. Uma estrutura inicial é:

```
Plain Text
```

```

function isSignedIn() {
  return request.auth != null;
}

function userRole() {
  return get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role;
}

function hasRole(roles) {
  return isSignedIn() && userRole() in roles;
}

function canManageCommercial() {
  return hasRole(['admin', 'gerente']);
}

```

Na prática, divida permissões por operação. Por exemplo, Comercial pode criar contratos, mas não pode registrar baixa; Financeiro pode alterar parcelas, mas não pode alterar limites do catálogo; Operador pode gerenciar anúncios, mas não pode ler valores financeiros. Também valide campos obrigatórios, tipos e chaves permitidas com `keys().hasOnly(...)` quando apropriado.

Para operações sensíveis, a regra deve permitir somente o resultado mínimo esperado ou bloquear a escrita direta e aceitar apenas uma API server-side. Como as bibliotecas administrativas de servidor contornam as Security Rules, o servidor precisa validar o token e o papel antes de escrever; não basta instalar o Admin SDK e confiar na chamada. 9

Crie `firestore.indexes.json` com índices baseados nas consultas reais. A lista inicial pode ser:

JSON

```

{
  "indexes": [
    {
      "collectionGroup": "contracts",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "status", "order": "ASCENDING" },
        { "fieldPath": "createdAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "installments",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "status", "order": "ASCENDING" },

```

```

    { "fieldPath": "dueDate", "order": "ASCENDING" }
  ],
},
{
  "collectionGroup": "client_ads",
  "queryScope": "COLLECTION",
  "fields": [
    { "fieldPath": "contractId", "order": "ASCENDING" },
    { "fieldPath": "format", "order": "ASCENDING" },
    { "fieldPath": "status", "order": "ASCENDING" }
  ]
}
],
"fieldOverrides": []
}

```

Ajuste os índices depois de observar as consultas reais. Não crie dezenas de combinações preventivamente; cada consulta que combina filtros e ordenação deve ser verificada no emulador e no projeto de homologação.

11. Dashboard e relatórios comerciais

Não remova `AdminAnalytics.tsx` : ele é o dashboard de audiência existente. Adicione um dashboard comercial separado ou crie uma navegação superior entre “Audiência” e “Comercial” . O novo dashboard deve consumir resumos e não recalculiar tudo no navegador.

Card	Origem recomendada	Regra
Clientes ativos	resumo ou contagem de <code>clients</code>	<code>status == active</code>
Contratos ativos	<code>contracts</code> /resumo	<code>status == active</code>
Valores a receber	resumo financeiro	parcelas não pagas
Recebido no mês	<code>financial_summaries/{YYYY-MM}</code>	pagamentos do mês
Pacotes mais vendidos	resumo ou contratos agrupados	contratos ativos/fechados

Implemente `AdminReports.tsx` com quatro relatórios pedidos: vendas por pacote, clientes por pacote, contratos por duração e faturamento por mês. Cada relatório deve permitir período, status e exportação CSV. O relatório precisa distinguir “valor contratado” de “valor recebido” ; contrato fechado não significa parcela paga.

Para não introduzir dependência de gráfico sem necessidade, reaproveite `recharts`, que já está no `package.json`. Use tabelas acessíveis além de gráficos: a tabela é a fonte de conferência para o financeiro.

12. Captação pública pela página `/anuncie`

Depois do módulo interno, altere `MidiaKitPage.tsx` em duas fases. Primeiro, substitua os textos e valores fixos por uma leitura pública dos pacotes ativos e uma pré-visualização semelhante à do painel. Segundo, implemente o formulário de solicitação comercial numa coleção `sales_leads` ou numa API de leads.

O formulário deve coletar nome, empresa, e-mail, telefone/WhatsApp, pacote de interesse, duração e mensagem. Ele não deve criar um cliente ou contrato automaticamente. O fluxo correto é lead recebido, análise comercial, cliente cadastrado e contrato fechado pela equipe.

Se a equipe quiser esconder preços, a página pública pode mostrar apenas benefícios e um botão “Solicitar proposta”. Se os preços forem públicos, libere somente pacotes `isActive == true` e nunca exponha campos internos como margem, custo, observações ou dados do cliente.

13. Seed e migração de dados

Crie `scripts/seed-commercial.mjs` com Firebase Admin SDK. O seed deve ser idempotente: use IDs estáveis como `bronze`, `prata`, `ouro` e `master`, ou consulte pelo `slug` antes de criar. Nunca duplique pacotes a cada execução.

O seed inicial deve conter benefícios de exemplo, mas os preços precisam ser preenchidos pelo Grupo Grapiúna. Os limites conhecidos podem começar assim:

Plain Text

```
const packages = {
  bronze: {
    name: 'Bronze',
    adLimits: { cover: 0, leaderboard: 0, intermediario: 0, sidebar: 1, mobile:
  },
  ouro: {
    name: 'Ouro',
    adLimits: { cover: 1, leaderboard: 2, intermediario: 0, sidebar: 2, mobile:
  },
};
```

Use `null` ou uma etapa de configuração para Prata e Master enquanto o comercial não definir os valores. Não publique um pacote ativo com preço zero por engano. O script pode falhar deliberadamente quando houver preço obrigatório ausente.

Crie `migrate-commercial.mjs` somente se houver dados antigos para importar. A coleção `ads` atual não deve ser convertida automaticamente em anúncio de cliente sem uma planilha de correspondência; ela provavelmente contém anúncios editoriais do portal. Faça backup/exportação antes de qualquer migração.

14. Testes obrigatórios

A validação precisa cobrir mais do que o formulário. O risco maior está em ultrapassar cotas, duplicar contratos e registrar pagamentos incorretos.

Tipo	Casos mínimos
Tipos e Zod	nome curto, cor inválida, preço negativo, limite decimal, duração inválida
Pacote	criar, editar, reordenar benefício, remover conteúdo, desativar pacote vendido
Preview	alternar 1/3/6/12 meses, mostrar preço e limites corretos, responsividade
Contrato	snapshot correto, datas de fim, 1/3/6/12 parcelas, clique duplo idempotente
Calendário	vencimento no dia 28, 29, 30 e 31; fevereiro e virada de ano
Cota	criar até o limite, rejeitar limite excedido, liberar após pausa, concorrência
Financeiro	baixa integral, parcial, atraso, estorno, pagamento duplicado
Papéis	cada papel acessa somente suas telas e operações
Regras	visitante não lê dados privados; usuário comum não grava catálogo/financeiro
Regressão	portal público, <code>AdBanner</code> , notícias, grade, upload e login Google

Use o Firebase Emulator Suite para testes de regras e dados, evitando testar escritas destrutivas diretamente na produção. Para cada regra de coleção nova, teste pelo menos leitura não autenticada, leitura autorizada, escrita autorizada e escrita com payload inválido.

Faça também uma verificação manual de ponta a ponta: criar Ouro, cadastrar benefício “Suporte por WhatsApp”, cadastrar conteúdo “Publicação de matéria”, definir 1 Capa TV, 2 Leaderboards, 2 Sidebars e 4 Mobiles, criar cliente, fechar 12 meses, conferir 12 parcelas, alocar os anúncios e registrar uma baixa.

15. Implantação

Adicione as variáveis server-side apenas no ambiente da Vercel ou no ambiente local seguro:

Plain Text

```
FIREBASE_PROJECT_ID=...  
FIREBASE_CLIENT_EMAIL=...  
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\\n...\\n-----END PRIVATE KEY-----"
```

O código de inicialização deve normalizar quebras de linha da chave privada com `replace(/\\n/g, '\\n')`. Não comite `.env.local`, service account JSON ou qualquer chave privada.

Publique as regras e índices somente depois de validar no emulador e em um projeto de homologação:

Bash

```
npx firebase emulators:start --only firestore,auth  
npm run lint  
npm run build  
npx firebase deploy --only firestore:rules,firestore:indexes  
npm run dev
```

O repositório atual usa `vercel.json` para reescrever a aplicação SPA e já mantém endpoints em `api/`. As novas rotas serverless devem seguir o padrão de exportação de `api/youtube/live.ts`, mas com autenticação e sem cache público para operações comerciais.

1 Após publicar, configure as variáveis de ambiente na Vercel, faça um contrato de teste com um usuário restrito e confirme que o portal público continua carregando os anúncios legados.

Atualize o README do repositório para refletir a arquitetura real. O README anexado não deve ser copiado sem adaptação, pois os comandos `npm run db:migrate`, `npm run db:seed` e a

configuração PostgreSQL não existem nesta base. 2

16. Critérios de aceite

A primeira entrega pode ser considerada concluída quando o administrador cria e edita Bronze, Prata, Ouro e Master sem código adicional, define preços por 1, 3, 6 e 12 meses, adiciona benefícios, escolhe serviços do catálogo e define limites por formato.

A segunda condição é que a pré-visualização mostre exatamente o preço, os benefícios, os conteúdos e os limites do período selecionado. A terceira é que o vendedor consiga cadastrar um cliente, fechar um contrato e gerar automaticamente o número correto de parcelas com snapshot do pacote.

A quarta condição é que o operador consiga alocar anúncios até a cota, receba erro ao tentar ultrapassá-la e publique somente anúncios aprovados. A quinta é que o financeiro consiga filtrar pendentes, pagos e atrasados e registrar baixas auditadas. A sexta é que relatórios apresentem valor contratado e valor recebido separadamente.

Por fim, nenhum papel não autorizado pode alterar pacote, contrato, limite, parcela ou usuário por chamada direta ao Firestore ou à API. O portal público existente, incluindo anúncios, notícias, grade e autenticação atual, deve continuar compilando e funcionando.

17. Alternativa caso PostgreSQL seja obrigatório

Se o requisito de PostgreSQL do README anexado for inegociável, não misture parte dos contratos no Firestore e parte no PostgreSQL sem um plano de sincronização. A alternativa limpa é manter o portal público React/Vite e criar um backend administrativo separado em Next.js ou em uma API Node, com PostgreSQL como fonte única do módulo comercial.

Nesse cenário, o modelo seria relacional: `packages`, `package_prices`, `benefits`, `package_benefits`, `service_catalog`, `package_services`, `clients`, `contracts`, `contract_benefits_snapshot`, `contract_services_snapshot`, `ad_formats`, `contract_ad_limits`, `client_ads`, `installments`, `payments`, `users` e `audit_logs`. A API server-side faria as transações de fechamento, baixa e cota; o frontend atual consumiria essa API.

A migração exigiria uma etapa adicional para autenticação, sessão HTTP-only, banco, ORM, migrações e deploy. Portanto, para entregar rapidamente o que foi pedido dentro deste repositório, a opção Firestore é a mais coerente; para atender literalmente a stack do README, o projeto deve ser tratado como uma reimplementação de backend e não como uma simples feature.

Referências

[1] Repositório Grupo Grapiúna no GitHub

- [2] package.json do repositório analisado
- [3] Painel administrativo atual
- [4] Cadastro atual de anúncios
- [5] Componente público de banners
- [6] Regras atuais do Firestore
- [7] Página pública atual de mídia kit
- [8] Cloud Firestore data model
- [9] Firebase role-based access control
- [10] Firebase transactions and batched writes
- [11] Firebase password authentication for web
- [12] Firebase Admin SDK user management